



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

CSS PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Web Technologies

TOTAL: 10 QUESTIONS

Q1 What is CSS and what problem in Web Technologies does it solve? **Model**

Q6 How does CSS handle reliability and high availability? **Observability**

Q2 What are the core components or concepts in CSS? **Architecture**

Q7 What observability does CSS provide out of the box? **Lifecycle**

Q3 How does CSS compare to its main alternatives? **Configuration**

Q8 What are common performance bottlenecks in CSS? **Compliance**

Q4 How do you get started with CSS for a new project? **Hardening**

Q9 What security best practices apply when running CSS? **Testing**

Q5 What configuration options most affect CSS's behavior in production? **ISO / IdP**

Q10 What mistakes do teams commonly make when adopting CSS? **CSS2**



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 CSS is a tool or platform that

Mitigation

CSS is a tool or platform that automates or simplifies a specific concern in the Web Technologies domain, reducing manual effort and operational complexity for engineering teams.

A6 CSS provides HA through leader election, replication, and observability

CSS provides HA through leader election, replication, or stateless horizontal scaling. Deploy at least two instances across availability zones and configure health checks for automatic failover.

A2 CSS has key building blocks that work

Primitives

CSS has key building blocks that work together: a configuration layer defines intent, a runtime layer enforces it, and an API or CLI layer allows operators to manage and observe the system.

A7 CSS exposes metrics (Prometheus endpoint or built-in dashboard) and automation

CSS exposes metrics (Prometheus endpoint or built-in dashboard), structured logs, and optionally distributed traces. Define SLOs on the key metrics and alert before users are affected.

A3 CSS trades off simplicity against feature richness

Hardening

CSS trades off simplicity against feature richness differently than its alternatives. Choose CSS when its specific strengths performance, ecosystem, or ease of use align with your team's primary constraints.

A8 Performance bottlenecks typically stem from misconfiguration and resource

Performance bottlenecks typically stem from misconfigured connection pools, large payload sizes, insufficient worker threads, or missing caches. Profile with built-in diagnostics before optimizing.

A4 Install CSS via its package manager or

Gotchas

Install CSS via its package manager or binary release. Follow the quickstart to initialize a project, configure the minimal required settings, and run the default command to verify the setup works end-to-end.

A9 Run CSS with the principle of least

Assessment

Run CSS with the principle of least privilege, enable TLS for all communication, use a secrets manager for credentials, and keep the software patched to the latest stable release.

A5 Production-critical settings typically include concurrency, replication, and

Configuration

Production-critical settings typically include concurrency limits, timeout values, retry policies, resource limits, and logging levels. Review the official production hardening guide before deploying.

A10 Common pitfalls include skipping the documentation and

Configuration

Common pitfalls include skipping the documentation and learning the API by trial and error, using default configurations in production, lacking a runbook for operational issues, and not testing failover scenarios.